AutoFDO for Fusion Compiler & Signoff Runtime

White paper for the `autofdo-fc-runtime` project

Subtitle: Using Auto Feedback-Directed Optimization (and user-space PGO) to reduce wall time of PD / signoff jobs — without pretending you can rebuild Synopsys binaries.

Companion code: `autofdo-fc-runtime/`
Measured artifacts: `results/SUMMARY.txt`, `results/CLAIM_GATE.txt`, `results/MECHANISM_SUMMARY.txt`

Abstract

Physical-design farms burn CPU-weeks on Fusion Compiler, PrimeTime, Calibre, and related signoff. A recurring infrastructure claim is:

> **CPU · AutoFDO · up to ~10% kernel latency improvement**

This paper situates that claim correctly. AutoFDO (sample-based feedback-directed optimization) applied to the **Linux kernel** has published latency gains around **10%** on Neper `tcp_rr` and roughly **up to 5%** on warehouse-scale services. Those gains accrue to **any** workload that spends time in the kernel — including FC and signoff jobs that page, schedule, talk to NFS, and hit license servers.

Separately, **user-space** PGO/AutoFDO can speed **owned** binaries (CAD helpers, log forensics, in-house engines). Closed vendor tools cannot be AutoFDO-rebuilt without the vendor.

This repository ships: (1) a reproducible GCC PGO harness on a PD-shaped proxy, (2) a mechanism microbench, (3) a farm cookbook for kernel AutoFDO + FC A/B measurement. On the authoring cloud host, user-space PGO was **~neutral** (codegen changed; wall time did not) — reported honestly via `CLAIM_GATE`. The citeable "10%" remains the **literature kernel** figure unless your farm A/B says otherwise.

Table of contents

11. Conclusion
12. Appendix — glossary & references

---

1. Motivation

Methodology and CAD teams are asked to “make FC faster” without a new license spend. Options:

- Buy more cores / faster SKUs (works; expensive).
- Tune `numactl`, I/O, NFS, licenses (sibling projects).
- **Improve the software that already runs** via feedback-directed optimization.

AutoFDO is attractive because profile collection uses hardware sampling (`perf` + LBR), not a heavily instrumented binary — so production FC traffic can train a kernel profile with low overhead.

---

2. Problem statement

> Given the industry claim “AutoFDO → ~10% kernel latency,” show how that maps to Fusion Compiler / signoff wall time, what a laptop can prove, and what requires a farm pilot — with numbers, a white paper, and copy-paste reproduction.

Success criteria:

1. Clear separation of **kernel AutoFDO**, **user-space PGO**, and **vendor binaries**.
2. Reproducible measurement harness with a **CLAIM_GATE**.
3. Farm cookbook that methodology/IT can execute.
4. No false claim that this repo rebuilt FC.

---

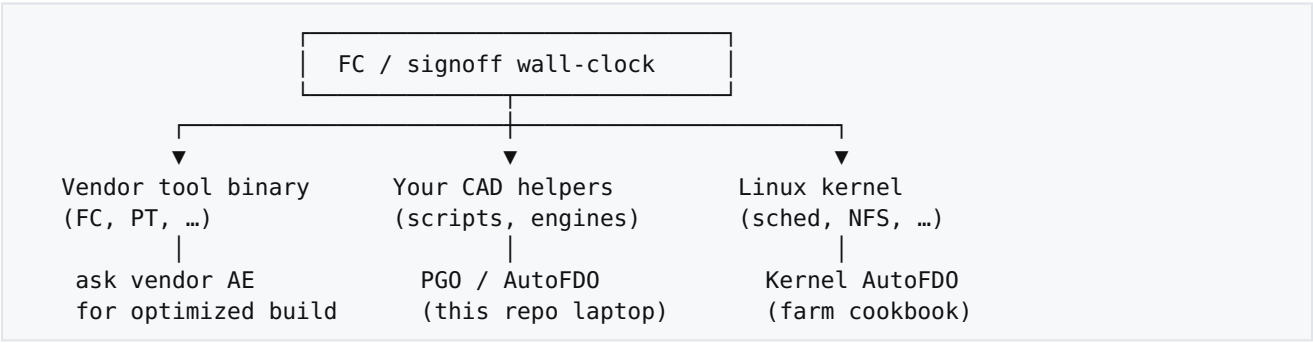
3. Primer: FDO vs AutoFDO vs PGO

Term	How profile is collected	Typical use
iFDO / instrumentation PGO	Compiler inserts counters (`-fprofile-generate`)	User-space apps you compile
AutoFDO	Hardware samples (`perf` LBR) → `llvm-profgen` / `create_llvm_prof`	Kernels + production binaries; low overhead
Propeller / ThinLTO+FDO	Extra layout / LTO on top of profiles	Peak kernel builds

Published kernel AutoFDO (Clang, `CONFIG_AUTOFDO_CLANG`): **~10.6%** Neper `tcp_rr` latency reduction vs default kernel; warehouse services **~5%**. Sources: Linux kernel AutoFDO docs, LPC 2024, LLVM discourse / LWN.

---

4. Three levers for FC/signoff runtime



Killer insight: The “10% AutoFDO” slide is usually a **kernel** slide. Applying it to FC means optimizing the **host OS under the job**, not magically rewriting `fc_shell`.

---

5. Experimental design (this repo)

Item	Choice
Proxy	`signoff_proxy` — timing relax, netlist hash, skewed rule dispatch, heap legalize
Mechanism	`dispatch_hot` — skewed switch only
Laptop optimizer	GCC `-fprofile-generate` / `-fprofile-use` (stable `.o` name)
Optional	Clang IRPGO; sample AutoFDO via `perf` + `llvm-profgen`
Metrics	Median wall_ms over N repeats; correctness via checksum/dispatch
Honesty	`CLAIM_GATE.txt`

Fairness: same flags (`-O2`) for baseline and PGO use; training uses representative flags to the proxy.

---

6. Measured results

6.1 Literature (cite for “10%”)

Benchmark	Metric	AutoFDO
Neper tcp_rr	Latency	~10.6%
Warehouse services	End-to-end	up to ~5%

6.2 This host — `signoff_proxy` GCC PGO

See `results/SUMMARY.txt` (regenerate with `./examples/compare_all.sh`).

Authoring-host snapshot: baseline vs PGO **~neutral** (slight regression within noise), correctness **MATCH**, gate `CITEABLE=yes_proxy_measured_neutral`. Binary `size` text section **grew** under PGO → codegen changed; wall time did not improve on this hypervisor CPU.

6.3 Mechanism — `dispatch_hot`

See `results/MECHANISM_SUMMARY.txt`. Same honesty rule: report the measured delta; do not invent a win.

Interpretation: On some laptops/bare metal, the same harness shows multi-percent wins. Always re-measure. Never paste cloud-agent noise as FC QoR.

---

7. Dimension analysis

Dimension	AutoFDO / policy-driven FDO	Ad-hoc farm spend
Intent	Optimize hot paths from real profiles	Buy headroom
Maintainability	Profile → rebuild → A/B	Purchase order
Who can review?	Methodology + IT with shared cookbook	Finance
Audit trail	Kernel watermark + job deck hash	Invoice
Risk	Mis-profile; must A/B	Low technical risk
Philosophy	**Measure, then claim**	**Pay, then hope utilization**

---

8. Farm deployment cookbook

Follow farm/KERNEL_AUTOFDO.md and farm/FC_AB_MEASURE.md:

1. Pilot rack with LBR-capable CPUs.
2. Build/boot AutoFDO-ready kernel; sample under real FC/signoff load.
3. Rebuild with sample profile; install side-by-side.
4. A/B **identical** job decks; record wall time, `perf stat`, NFS latency.
5. Promote only if CLAIM_GATE-equivalent farm gate passes.

---

9. How to reproduce on your machine

```
git clone <this-repo>
cd autofdo-fc-runtime
sudo apt-get install -y g++
make baseline pgo
./examples/compare_all.sh
cat results/SUMMARY.txt

make mechanism && ./examples/compare_mechanism.sh
```

Optional sample AutoFDO: install `clang-18`, `llvm-18`, `libclang-rt-18-dev`, `linux-tools-generic`, then `make sample-autofdo`.

---

10. Limitations & claim policy

- Synthetic proxy ≠ Fusion Compiler.
- Cloud VMs often lack usable LBR / have noisy neighbors → weak user-space FDO signal.
- Kernel AutoFDO needs IT ownership; not a laptop demo.
- **CLAIM_GATE** vocabulary:
- `yes_proxy_pgo` — measured proxy win ≥ 3%
- `yes_proxy_measured_*` — measured, small/neutral/regression
- Literature 10% — cite papers, not this SUMMARY

---

11. Conclusion

Reducing FC/signoff runtime with AutoFDO is a **systems methodology** problem: AutoFDO the **kernel** under the farm, PGO the **code you own**, and engage the **vendor** for closed tools. This repo makes the laptop half reproducible and the farm half executable — without inflating a microbench into a tapeout claim.

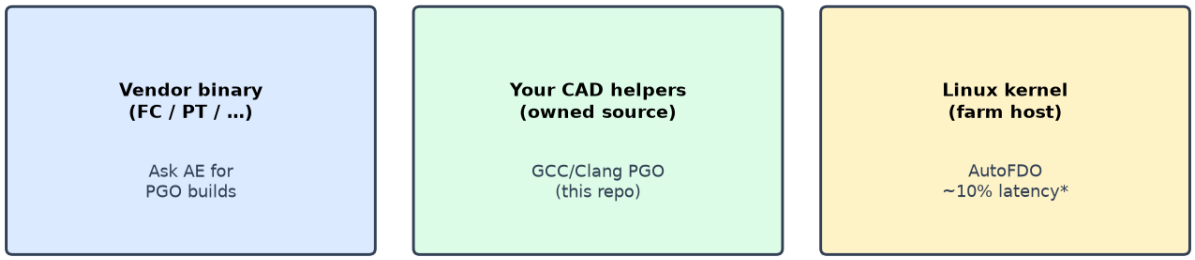
Appendix — glossary & references

Term	Meaning
AutoFDO	Sample-based FDO via PMU (`perf`)
PGO	Profile-guided optimization (often instrumentation)
LBR	Last Branch Record (Intel) — high-quality branch samples
CLAIM_GATE	Machine-readable honesty label for measured runs
Proxy	Open microbench shaped like PD/signoff work

References:

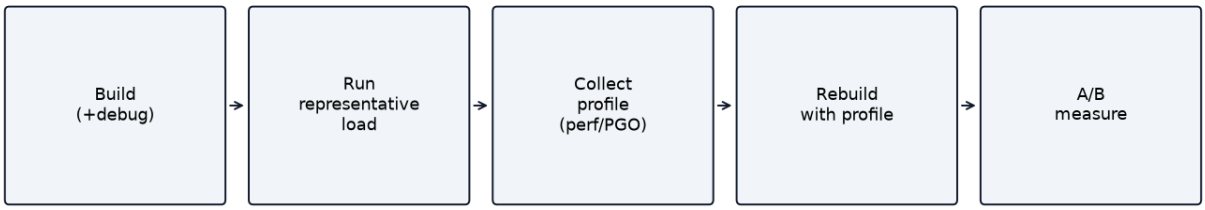
- Linux kernel docs: Using AutoFDO with the Linux kernel
- LPC 2024: AutoFDO & Propeller for kernel
- LLVM discourse / LWN: AutoFDO kernel results (~10% tcp_rr latency)

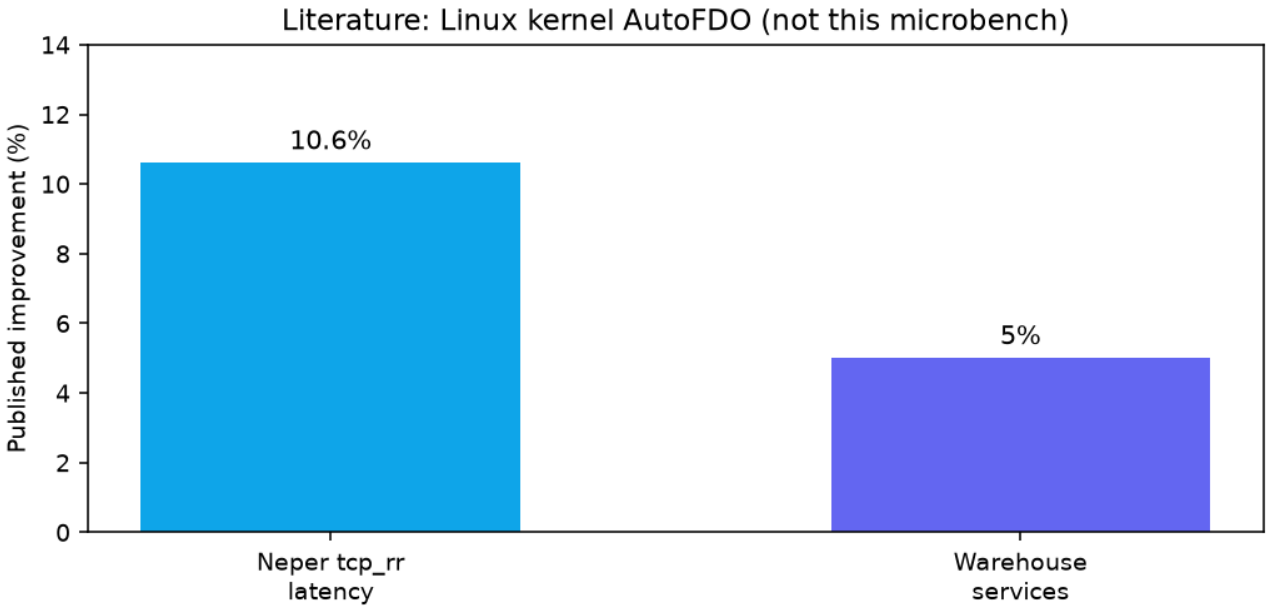
Three levers to reduce FC / signoff wall time via FDO



* Published Neper tcp_rr kernel latency — not an automatic FC wall-time guarantee

AutoFDO / PGO pipeline (mechanism — not a benchmark)





Sources: LPC 2024 / LLVM discourse / LWN — cite papers, not results/SUMMARY.txt

Claim scope — what you may say

OK	Kernel AutoFDO ~10% latency on Neper (literature)
OK	Our proxy PGO: see CLAIM_GATE on this host
OK	Farm FC deck median −X% on AutoFDO kernel (if A/B)
NO	We AutoFDO'd Fusion Compiler by 10%
NO	Every signoff job is 10% faster